МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1 по курсу «Компьютерная графика»
Основы 3D графики

Вариант 1 — Куб

Выполнил: Граховский Сергей Константинович
Группа: М8О-305БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Цель работы

Построить простой трёхмерный объект (куб) и изучить его отображение на плоскость экрана с помощью матриц проекции и аффинных преобразований. Программа работает в реальном времени. Для реализации использованы C++, Vulkan, GLFW и ImGui.

В программе выполнено базовое задание: отрисовка куба с учётом проекции и аффинных преобразований, без дополнительных заданий.

Метод решения

Построение куба

Фигура задана в локальной системе координат, то есть до перемещения и поворота. Её центр находится в начале координат. Куб имеет 8 вершин, расположенных в углах куба со стороной 2 (координаты ±1 по каждой оси):

(+1, +1, +1), (+1, +1, −1), (+1, −1, +1), (+1, −1, −1),
(−1, +1, +1), (−1, +1, −1), (−1, −1, +1), (−1, −1, −1)

Все рёбра куба имеют одинаковую длину, все грани — квадраты, разбитые на два треугольника. У куба 6 граней, каждая из которых разбита на 2 треугольника. В коде используется индексный буфер: 6 граней по 2 треугольника по 3 вершины дают 36 индексов.

Структура вершины содержит позицию и цвет:

cpp

```
struct Vertex {
    glm::vec3 pos;
    glm::vec3 color;
};
```

Цвет каждой вершины задан вручную — разный для наглядности. Например, вершина в переднем левом нижнем углу красная, в переднем правом нижнем — зелёная. За счёт интерполяции цвета между вершинами грани получаются разноцветными.

Индексы задают порядок соединения вершин в треугольники:

cpp

```
const std::vector<uint32_t> indices = {
    0, 1, 2,  2, 3, 0,   // front
    1, 5, 6,  6, 2, 1,   // right
    5, 4, 7,  7, 6, 5,   // back
    4, 0, 3,  3, 7, 4,   // left
```

```
    3, 2, 6,  6, 7, 3,   // top
    4, 5, 1,  1, 0, 4,   // bottom
};
```

Это 12 групп по 3 индекса — двенадцать треугольных граней.

Однородные координаты и матрицы

Обычная точка в пространстве задаётся тремя координатами (x, y, z). Для преобразований в программе к ним добавляется четвёртая компонента w. Исходная точка записывается как столбец (x, y, z, 1). Такая запись называется однородными координатами.

Дополнительная компонента позволяет записать перенос как умножение на матрицу. Матрица 3 на 3 подходит для поворота и масштабирования относительно начала координат, но не может добавить к каждой точке постоянное смещение. Матрица 4 на 4 объединяет эти операции в одной форме.

Масштабирование, поворот и перенос

Масштабирование задаётся диагональной матрицей 4 на 4: координаты умножаются на коэффициенты sx, sy, sz. Поворот задаётся углом и осью: в программе используются матрицы поворота вокруг осей X, Y и Z, построенные через синус и косинус угла. Перенос задаётся матрицей, у которой последний столбец содержит смещение (tx, ty, tz).

В программе для построения матриц используется библиотека GLM: функции glm::translate, glm::rotate, glm::scale, glm::perspective, glm::lookAt. Это стандартные функции для работы с матрицами 4 на 4.

Порядок преобразований

Матрица модели строится как произведение M = T * R * S, где T — перенос, R — поворот, S — масштабирование. В текущей версии программы перенос и масштабирование задаются пользователем через ImGui:

cpp

```
model = glm::translate(glm::mat4(1.0f), position);
model = glm::rotate(model, glm::radians(rotation.x), glm::vec3(1, 0, 0));
model = glm::rotate(model, glm::radians(rotation.y), glm::vec3(0, 1, 0));
model = glm::rotate(model, glm::radians(rotation.z), glm::vec3(0, 0, 1));
model = glm::scale(model, scale);
```

При умножении на вектор-столбец операции выполняются справа налево: сначала масштабирование, затем поворот, затем перенос. Порядок важен:

если сначала повернуть, а затем масштабировать неравномерно, фигура деформируется.

Камера и матрица вида

Камера фиксирована в точке (0, 0, 6) и направлена в начало координат. Матрица вида V строится функцией glm::lookAt:

cpp

```
view = glm::lookAt(glm::vec3(0.0f, 0.0f, 6.0f),
                   glm::vec3(0.0f),
                   glm::vec3(0.0f, 1.0f, 0.0f));
```

Она переводит мировые координаты в систему координат камеры. Вектор (0, 1, 0) задаёт направление «вверх» на экране.

Перспективная проекция

Перспективная проекция создаёт эффект глубины: более удалённые объекты выглядят меньше. В коде вертикальный угол обзора равен 45 градусов, ближняя плоскость n = 0.1, дальняя плоскость F = 100. Матрица проекции строится функцией glm::perspective:

cpp

```
proj = glm::perspective(glm::radians(45.0f), aspect, 0.1f, 100.0f);
proj[1][1] *= -1.0f;
```

Параметр aspect — отношение ширины окна к высоте. Он пересчитывается каждый кадр, чтобы фигура не растягивалась при изменении размеров окна. Строка proj[1][1] *= -1.0f инвертирует ось Y, так как в Vulkan координаты NDC направлены иначе, чем в OpenGL.

В вершинном шейдере итоговая матрица применяется к каждой вершине:

glsl

```
gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition, 1.0);
```

Результат — координаты отсечения. Затем графический конвейер выполняет деление x, y и z на w и преобразует координаты в положение на экране.

Передача матриц в шейдер

Матрицы model, view и proj передаются в шейдер через uniform-буфер. Это три матрицы 4 на 4 — 192 байта. Они обновляются каждый кадр: model отвечает за перенос, поворот и масштабирование фигуры, view задаёт положение камеры, proj — перспективную проекцию.

cpp

```
struct UniformBufferObject {
    glm::mat4 model;
    glm::mat4 view;
    glm::mat4 proj;
};
```

Буфер создаётся с флагами VMA_ALLOCATION_CREATE_HOST_ACCESS_SEQUENTIAL_WRITE_BIT | VMA_ALLOCATION_CREATE_MAPPED_BIT, поэтому он постоянно замаплен на CPU, и в него можно писать через memcpy каждый кадр.

Объекты Vulkan и отрисовка

Координаты и цвета вершин записываются в вершинный буфер VkBuffer, а номера вершин треугольников — в индексный буфер. Память буферов выделяется с помощью VMA.

Графический конвейер VkPipeline настроен на список треугольников. Вершинный шейдер преобразует координаты и передаёт цвет вершины. Фрагментный шейдер выводит интерполированный цвет. Проверка глубины позволяет ближним поверхностям перекрывать дальние независимо от порядка отрисовки.

Работа программы разделена на этапы: initialize создаёт ресурсы; update обрабатывает интерфейс и обновляет матрицы; render записывает данные и команды отрисовки; shutdown освобождает ресурсы.

GLFW используется для создания окна и обработки событий, а ImGui — для элементов управления. В окне ImGui отображаются текущие параметры трансформации куба: позиция, поворот и масштаб, а также кнопка сброса.

Функция render записывает команды в командный буфер:

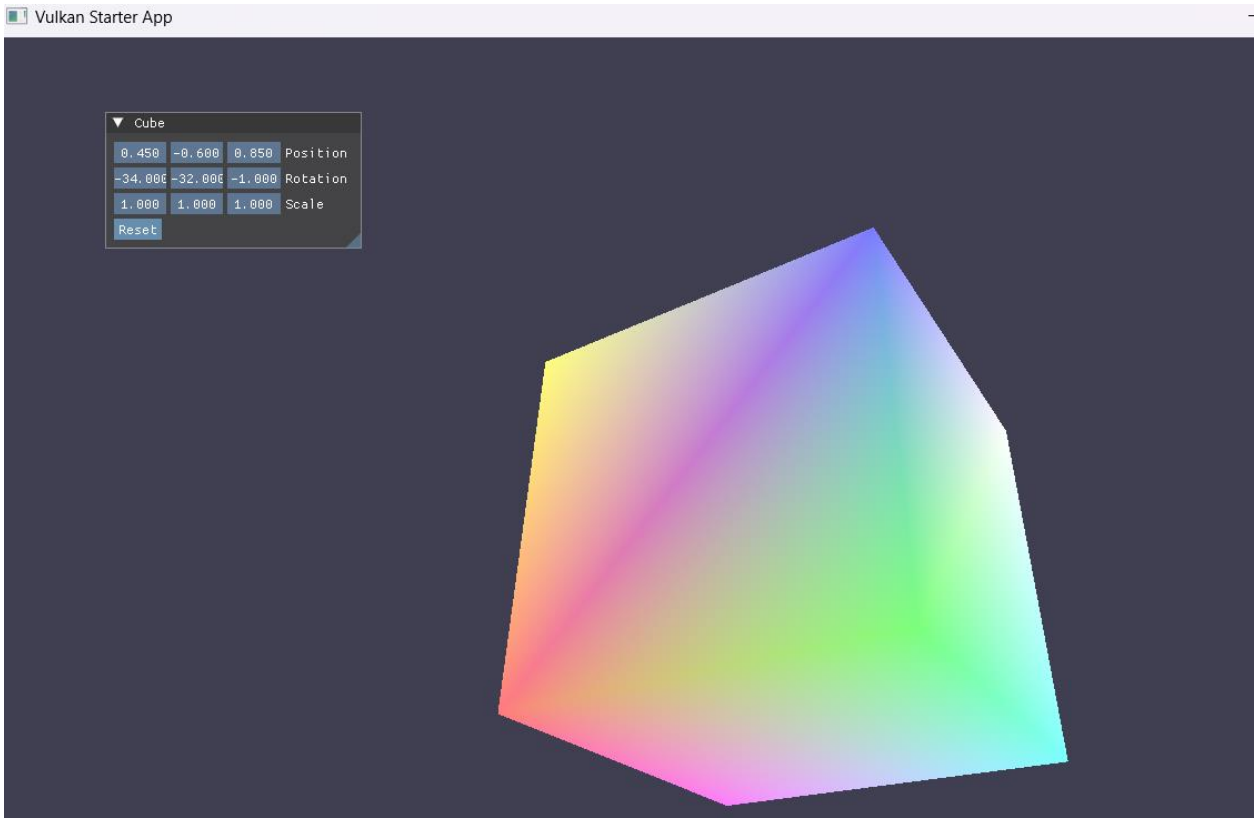
cpp

```
vkBeginCommandBuffer(fd.command_buffer, &begin_info);
vkCmdBeginRenderPass(fd.command_buffer, &render_pass_info,
VK_SUBPASS_CONTENTS_INLINE);
vkCmdBindPipeline(fd.command_buffer,
VK_PIPELINE_BIND_POINT_GRAPHICS, pipeline);
vkCmdBindVertexBuffers(fd.command_buffer, 0, 1, &vertex_buffer, &offset);
vkCmdBindIndexBuffer(fd.command_buffer, index_buffer, 0,
VK_INDEX_TYPE_UINT32);
vkCmdBindDescriptorSets(...);
vkCmdDrawIndexed(fd.command_buffer, 36, 1, 0, 0, 0);
```

```
vkCmdEndRenderPass(fd.command_buffer);
vkEndCommandBuffer(fd.command_buffer);
```

Функция vkCmdDrawIndexed рисует 36 индексов — двенадцать треугольников куба.

Результаты



На экране отображается куб в перспективной проекции. Видны разноцветные грани, цвет плавно интерполируется между вершинами. Пользователь может через ImGui-интерфейс изменять положение, поворот и масштаб куба; изменения применяются немедленно, так как матрицы пересчитываются каждый кадр. Кнопка Reset возвращает куб в исходное состояние.

Вывод

В ходе работы был построен трёхмерный объект (куб) и изучено его отображение на плоскость экрана с помощью матриц проекции и аффинных преобразований. Освоены базовые приёмы работы с Vulkan: создание вершинного и индексного буферов, uniform-буфера для матриц, графического конвейера с проверкой глубины, а также взаимодействие с пользователем через ImGui.